AI Code Translation for CodeCollab

A Python/JS/TS-to-Rust Performance Migration Assistant

Staff Product Manager Case Study

Jason Wu

March 2026

The Opportunity: Why AI Code Translation, Why Now

10-100x

Performance gains from Python/JS → Rust migrations

45%

of organizations now use Rust in production

0

mature AI tools for Python/JS → Rust migration

Proven by:

- Pydantic v2 (17x faster)
- Ruff (100x faster)
- Discord (eliminated GC spikes)
- Cloudflare Pingora (70% less CPU)
- Figma (10x serialization)

Sources: Rust 2025 Survey, Pydantic/Ruff/Discord/Cloudflare/Figma engineering blogs

Stated Assumptions

Market	Rust adoption continues accelerating (2.26M devs, 33% YoY growth). Performance-critical Python/JS services are growing faster than Rust talent supply. Enterprises will pay for tooling that de-risks Rust adoption.
Technical	Specialized harnesses over generalized LLM model improvements. LLM harnesses will be able to generate idiomatic Rust from Python/JS at method/module level with iterative repair. PyO3 and Neon FFI bindings are stable enough for production use.
Product	Developers will trust AI-generated Rust if given strong validation evidence with behavioral equivalence artifacts via differential/property testing. Reviewer bottleneck (~2x longer review times for AI code) is the primary adoption barrier.
Platform	AI agents are first-class participants in the development workflow. An API-first, MCP-compatible architecture is the right distribution model. GitHub Actions-style compute is available for running translation jobs.

Personas & Jobs To Be Done

Victoria — Performance Engineer

Senior Python/TS dev, 6 years exp

JTBD

Rewrite hot-path modules in Rust to eliminate latency bottlenecks — without becoming a Rust expert

KEY PAIN

Manual rewrites take months. Learning Rust's ownership model is a 3-4 month investment before productivity.

John — Staff Engineer / Gatekeeper

Migration approval authority, Rust-literate

JTBD

Approve migration PRs quickly and confidently — without spending days reviewing AI-generated code he can't trust

KEY PAIN

PR volume is surging 100% already, and AI generated code in some cases can take nearly double the time to review. He's the bottleneck.

Agent — Autonomous Maintainer

CI/CD-integrated translation agent

JTBD

Continuously identify optimization candidates, generate translations, and submit validated PRs — with no human initiation

KEY PAIN

Existing agents (Copilot, Devin) are general-purpose. No agent specializes in cross-language performance migration.

Strategic Scoping: Why This Problem, Not Others

COBOL → Java

RED OCEAN

127+ vendors (AWS, IBM, Google, TCS, Capgemini, 15+ specialists). $8.4B market dominated by hyperscalers with proprietary training data. Mission-critical banking systems where trust requires decades of mainframe expertise. TSB Bank disaster: £330M losses from failed migration.

C/C++ → Rust

DARPA-HARD

DARPA's TRACTOR program: $5M+ research grants, explicitly a research problem. 79% of C raw pointers can't translate to safe Rust. Undefined behavior, preprocessor macros, type punning have no Rust equivalent. Best tools: 22–48% success on small codebases. Google & Microsoft investing $1M+ each in interop, not translation.

Python/JS/TS → Rust

ZERO INCUMBENTS

No pointer arithmetic, no undefined behavior, no manual memory management — avoids hardest unsolved problems. Mature FFI: PyO3 (15.4K stars), Neon (8K stars), wasm-bindgen (225M+ downloads). Proven 10-100x gains. 45% org Rust adoption. Talent gap = tooling demand. Incremental: migrate hot paths only, not whole codebases.

Proposed Solution: The Migration Assistant

1

Profile

Analyze Python/TS codebase to identify hot paths via CPU/memory profiling integration. Surface candidates ranked by optimization potential.

→

2

Generate

Produce idiomatic Rust equivalents for targeted modules with FFI bindings (PyO3 for Python, Neon for JS/TS). Not line-by-line — semantic translation.

→

3

Validate

Auto-generate test harnesses. Run differential testing (same inputs ⟶ same outputs). Property-based testing via Hypothesis + proptest. Clippy for idiomaticity.

→

4

Ship

Submit PR with: Rust module, FFI bindings, test harness, benchmark results, and a reviewer-friendly diff showing behavioral equivalence evidence.

Technical Architecture: Agent-First, API-First

Agent Interface Layer

MCP-compatible server · AGENTS.md configuration · GitHub Actions execution · Issue/PR triggers · Enterprise audit logs

Core Translation Pipeline

Profiler Integration	LLM Translation + Repair Loop	Differential Testing Engine	PR Assembly + Benchmarks

Validation & Trust Layer

Property-based testing (Hypothesis ↔ proptest) · Clippy idiomaticity scoring · cargo-audit · Behavioral equivalence certificates · Performance proof

API-First Foundation: REST + MCP endpoints · Webhook events · Streaming status · Extensible via custom MCP tools

Validation: Earning Trust at Every Layer

Tier 1: Every PR

< 10 min

Compilation + Clippy (zero warnings target) · Unit test pass rate · 5-minute fuzz run · unsafe block count = 0

Tier 2: Nightly

< 60 min

Cross-language differential tests (same inputs → same outputs) · Property-based tests (Hypothesis ↔ proptest) · Extended input space coverage

Tier 3: Weekly/Release

< 4 hrs

Extended fuzzing campaigns · criterion.rs performance benchmarks vs. Python/JS baseline · cargo-audit dependency scan · Kani bounded model checking

Solving the Reviewer Bottleneck

Every PR includes: behavioral equivalence certificate · Clippy idiomaticity score · benchmark proof (Rust vs. baseline) · test coverage report · zero unsafe blocks. Target: 79%+ first-pass approval rate (matching Amazon's Java 8->17 benchmark).

Driving Alignment Across the Organization

Where Misalignment Typically Happens

Eng ↔ PM	"How accurate is 'accurate enough'?" Engineers want 100%; PM needs shippable thresholds.
Eng ↔ GTM	"Is this a feature or a product?" GTM wants to sell migration; Eng sees a targeted tool.
Leadership ↔ All	"When does this pay for itself?" Leadership wants ROI; teams want to ship and learn.

How to Resolve It

Shared success criteria doc
One page: accuracy thresholds, safety gates, launch criteria. Signed off by Eng, PM, Security, GTM before Phase 1.

Weekly product reviews
Bring GTM into reviews to be on the same page with value. Live demo of actual translation, beta feedback, case studies. GLM backed by reality.

Phase-gated decisions
Each phase has explicit go/no-go criteria. No ambiguity about what 'ready' means. If we need to adapt, I'll bring options based on our shared context.

My alignment philosophy: Bring everyone on the same product journey, with a plan that makes implicit judgements explicit. Write down everything — about quality bars, timelines, or definitions of success – get sign off, and keep everyone informed about the learnings along the way. This way we can either reference decisions, or adapt with shared context.
(e.g. accelerate commercial signal, or narrow scope to ship faster)

Rollout: Ship to Learn, Phase by Phase

Phase 1: Prove It Works

Months 1–3

Scope: Python → Rust only. Single-function hot paths. Internal dogfood (3-5 teams). CLI + API interface.
Goal: Validate core translation quality. Establish accuracy baselines. Learn what breaks.
Go/No-Go: ≥70% compilation rate after repair loop, ≥3 teams with a merged migration PR, NPS >30.

Phase 2: Build Trust

Months 4–7

Scope: Multi-function modules. Beta (20-50 teams). Full validation suite ships. Focus: Python → Rust depth over breadth.
Goal: Prove the reviewer experience. Reach 75%+ first-pass approval. Build confidence.
Go/No-Go: ≥75% first-pass approval, <15 min median time-to-first-translation, WAU growing MoM.

Phase 2b: Expand Language

Months 6–8

Scope: Add JS/TS → Rust (Neon bindings). Agent mode: issue-triggered translations on CI/CD.
Goal: Validate second language pair. Ship agent-first workflows.
Go/No-Go: JS/TS translation quality within 10% of Python baseline.

Phase 3: Scale & Monetize

Months 9–14

Scope: GA. MCP Registry listing. Enterprise tier (audit, compliance, SSO). Autonomous agent mode.
Goal: Drive adoption, org expansion, revenue. Establish the paved path for Rust migration.
Go/No-Go: ≥25% shipped translation rate, Net Revenue Retention >120%, enterprise pipeline >$2M Annual Recurring Revenue.

Measuring Success: Leading & Lagging Indicators

North Star Metric:

Total developer-hours saved through AI-assisted Rust migration

Leading Indicators (weekly/monthly)

- Translation compilation rate (target: >85%)
- Time-to-first-translation (target: <30 min)
- Weekly active translators (growing MoM)
- Repair loop convergence rate (improving)
- Clippy warning density (decreasing)

Lagging Indicators (quarterly)

- First-pass reviewer approval rate (target: >79%)
- Developer NPS (target: >50)
- Infrastructure cost reduction ($, measurable)
- Performance improvement (latency, throughput)
- Org expansion rate (individual → team → enterprise)

Baselining approach:

Benchmark manual migration effort before tool introduction. Industry data: manual dependency migration = 1+ dev-day per dependency (Amazon). Fully loaded senior eng = $75–200/hr.

Risks & Mitigations

Translation quality falls below threshold	**High**	Iterative repair loops double success rates. Fine-tuning on PyO3/Neon codebases. Phase 1 explicitly tests this before scaling. Exit criteria: if <50% after repair loop, pause and retrain.
Developers don't trust AI-generated Rust	**High**	Validation suite provides machine-verifiable evidence. Start with low-stakes modules (logging, serialization) where gains are visible and risk is low. Build trust incrementally.
Rust talent gap limits reviewer capacity	**Medium**	AI-assisted review (Clippy + automated checks) reduces expert burden. Pair reviews (Rust-expert + domain-expert). Training program alongside the tool.
LLM capabilities plateau or regress	**Medium**	Architecture is model-agnostic — swap underlying LLM without changing pipeline. Maintain evaluation benchmarks across model versions. Invest in fine-tuning as hedge.
Platform dependency (MCP/Actions instability)	**Low**	CLI-first architecture means core pipeline works without platform integration. MCP is an enhancement, not a requirement. Degrade gracefully.

Summary

Problem	AI code translation has failed at cross-language scale (8% accuracy). GitHub sunset its only attempt. But same-language upgrades prove the approach works (79% approval, $260M saved).
Insight	Python/JS→Rust is a solvable, underserved niche: no undefined behavior, mature FFI ecosystem, no mature incumbents, proven 10-100x gains, and a growing talent gap that creates demand.
Solution	A 4-step migration assistant (Profile → Generate → Validate → Ship) delivered as an MCP-compatible agent. — a performance migration paved path. Every PR includes machine-verifiable behavioral equivalence, benchmark proof, and idiomaticity scores.
Edge	Our advantage is our integrated pipeline and the data flywheel that builds over time.
Approach	Ship to learn in 3 phases. Explicit go/no-go gates. Alignment through shared success criteria. Measure with pipeline metrics, and satisfaction metrics. Target 79% first-pass approval.

Thank you — I'm looking forward to the discussion.

Questions?